до 11 янв., 01:30 | Открыть урок

Когда: 07.10.26 в 16:00

Первый контакт: Python как стиль жизни

- Что такое программа. Языки программирования
- IDE — первоначальная настройка
- Правила оформления кода PEP8
- Функция печати
- Переменные
- Ввод значений
- Немного о строках

«Лень, нетерпение и гордыня — три великие добродетели программиста».

Ларри Уолл

(создатель языка программирования Perl)

🧠 Что такое программа, или Как заставить железку думать

(спойлер: Сама по себе она не думает. Вообще. Никогда. Же-е-есть)

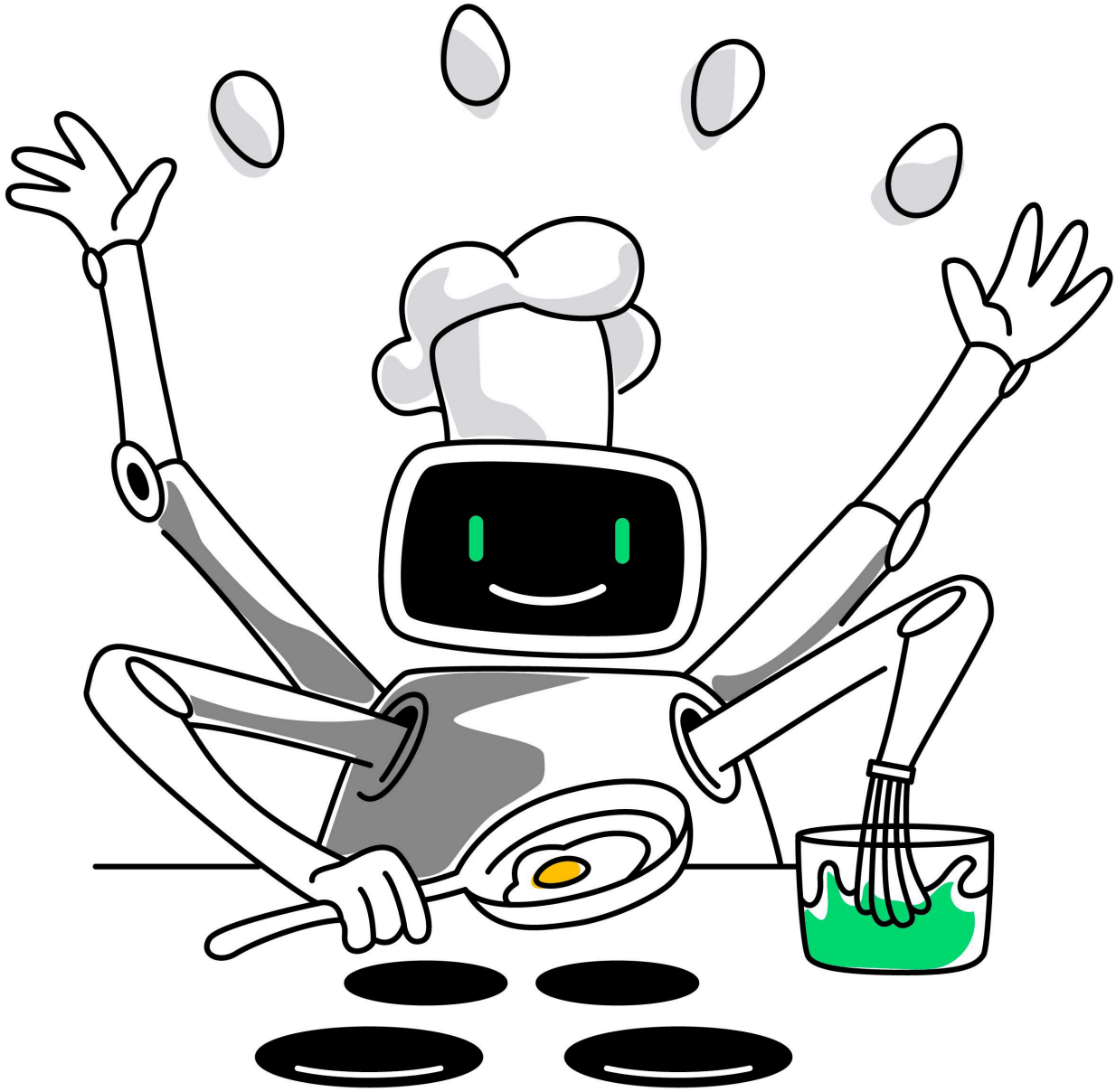
В компьютере много разных программ: браузер показывает нам котиков в интернете, медиаплеер играет любимые песенки, игры развлекают нас и делают жизнь лучше, а офисный пакет и вирусы — хуже. (Вирусы — специально, офисный пакет — нет 😬)

Чтобы котики появились в интернете, а музыка — на стриминговом сайте, тоже используются программы — графические или музыкальные редакторы, а также серверные приложения, которые и отправляют нам залипательный контент по запросу.

Хорошая новость: создание программ — не какой-то секретный магический ритуал, поэтому ничто, кроме нашей лени, не мешает нам заняться програ
прямо сейчас.

Если говорить чуть формальнее, то *программа — это набор инструкций, понятных компьютеру* (или любому другому устройству, которое может выполнять программы).

Программа — это как рецепт для робота-кулинара. Представьте: вы пишете инструкцию: «Возьми яйца, взбей, добавь сахар, испеки». Если робот туповат (а они все такие), он сделает именно это — даже если забыли упомянуть, что яйца нужно разбить, а не кинуть в миску целиком. Программирование — это искусство писать инструкции так, чтобы даже самый буквоедский железный ящик понял, чего вы от него хотите.



Вообще, родной язык для компьютеров — двоичный код. Он классный, но для человека не очень удобный. Потому что даже программа, которая выводит строку в консоль, выглядела бы как тысячи строк из нулей и единиц. А для её написания нам надо было бы изучить инструкцию к процессору на 3000 страниц.

Думаем, что вы догадались, что писать на двоичном коде примерно так же эффективно, как оперировать сердце лопатой 💔

Когда компьютеры можно было пересчитать по пальцам одной руки, стало очевидно, что нужен некоторый язык-посредник между человеком и устройством, который могли бы воспринимать обе стороны диалога. Так начали появляться первые языки программирования.

🐍 Языки программирования: магия для своих

Немного истории:

- 1940-е: Программисты тыкали проводами в лампочки (машинный код) и были полубогами.
- 1950-е: Ассемблер. Чуть понятнее, но всё равно выглядело как `MOV AX, 42`.
- 1970-е: C. «Пиши как человек!» — сказали разработчики и создали язык, с помощью которого можно было стрелять себе в ногу.
- 1991: Гвидо ван Россум создал Python, назвав его в честь «Летающего цирка Монти Пайтона». Теперь это язык, на котором можно написать что угодно — от скрипта для автоматизации скучных задач до нейросети, которая генерирует котиков в стиле барокко.

С момента появления первого языка программирования прошло уже много лет. И за это время появилось (и исчезло) довольно большое число разных способов заставить бездушную железку подчиняться нашим желаниям.

Если сильно упрощать, то дожившие до нас языки программирования бывают двух основных типов: компилируемые и интерпретируемые (а некоторые пытаются даже сразу усидеть на двух стульях).

Программы на компилируемом языке перед выполнением нужно сначала полностью проверить на некоторые ошибки и перевести в более понятную для компьютера форму. Это делает специальная программа — компилятор.

Обычно на компилируемых языках дольше разрабатывать, но сами программы работают быстрее.

Допиливаешь деталь напильником, пока она не встанет идеально в нужное место.

Если программа написана на интерпретируемом языке (JavaScript, Python), она не переводится целиком в машинный код, а специальная программа — интерпретатор — идёт по коду, анализирует и выполняет каждую отдельную команду.

Тут идеальный мир скорости и простоты разработки немного омрачает более медленная скорость работы результата.

Собираешь убер-багги, как орк во вселенной Вахи, и оно работает на магии веры того, что оно будет работать (до первой ошибки исполнения).

Компиляция vs интерпретация:

- **Компилируемые языки** (C++, Java): Вы пишете код → *компилятор* переводит его в машинный язык (или промежуточный код в случае с Java) раз и навсегда → программа запускается. Это как издать книгу — после печати изменить текст нельзя, только выпустить следующее издание, исправленное и дополненное.
- **Интерпретируемые языки** (Python, JavaScript): *Интерпретатор* переводит код построчно во время выполнения. Это как стендап — актёры (интерпретатор) импровизируют на ходу, и вы можете менять сценарий прямо во время действия.

КОМПИЛЯЦИЯ VS ИНТЕРПРЕТАЦИЯ







Почему же мы решили начать с Python? Есть несколько причин:

- Лёгкий синтаксис, можно читать почти как текст на английском языке.
- Универсальность: веб, данные, AI, скрипты для автоматизации рутины — везде засветился.
- Огромное сообщество. Если вы застряли, какой-нибудь «Петя из Твиттера» уже решил вашу проблему.

🛠 IDE — ваша цифровая мастерская

Исходный код программы на любом языке программирования — обычный текстовый файл, просто с необычным разрешением. Такой договор, что программы на Python принято писать в файлах с расширением .py (читается «пай», как «пирог» на английском).

Раз файлы текстовые, то команды для интерпретатора можно писать в обычном «Блокноте», но в жизни так делают разве что только настоящие фанаты «Блокнота». Для разработки чаще всего пользуются либо текстовыми редакторами с подсветкой синтаксиса выбранного языка программирования, либо ещё более мощными специальными программами, которые называются средами разработки (англ. IDE, Integrated Development Environment).

Среда разработки — тоже текстовый редактор, но с дополнительными возможностями. Например, она умеет сама находить на компьютере программу-интерпретатор и запускать всё одной кнопкой. Среда разработки, кроме того, форматирует написанный вами код, чтобы его удобно было читать, а иногда даже подсказывает, где вы допустили ошибку. Для Python есть довольно много разных бесплатных и платных инструментов, вот несколько из них:

- Wing IDE — неплохая учебная среда, но не слишком хорошо подходит для настоящих проектов. К тому же на Windows есть разные приключения с кодировками.
- PyCharm — для тех, кто любит, чтобы IDE думала за них (и ела 4 ГБ оперативки на завтрак).
- Visual Studio Code — выглядит просто, но настраивается невероятно гибко, а в умелых руках превращается в настоящий космолёт. К тому же не требует компьютера, на котором можно поиграть в GTA VI в 8K. Именно в VSCode мы и будем работать в этом курсе.

Мы тут никого не осудим, если вы захотите писать код в другой IDE, в онлайн-редакторе или даже в уже упомянутом «Блокноте» — программирование вообще не терпит никакой дискриминации. Но скриншоты и горячие клавиши будут рассказаны на примере VSCode ¯_(ツ)_/¯

🚀 Первоначальная настройка, или Готовимся к полёту

Для обучения нам понадобятся три вещи:

- собственно сам интерпретатор Python,
- среда разработки VSCode,
- понимание, как сдавать задачи.

1. Устанавливаем Python с python.org. Важно: галочка *Add Python to PATH* должна быть включена! Иначе потом придется шаманить с настройками. Linux/MacOS-юзерам: Расслабьтесь. Python у вас уже есть. Как и чувство превосходства над Windows-пользователями. На Windows 11 можно открыть приложение командной строки

и написать там `python` — тогда вас перенаправит в магазин приложений на установку Python, но там может быть более старая версия, чем на официальном сайте.

Примеры в курсе написаны с учётом версии интерпретатора `3.13` — если ваша версия меньше, то лучше обновиться, если больше, то всё должно работать.

2. Качаем VSCode с официального сайта. Устанавливаем. Запускаем.

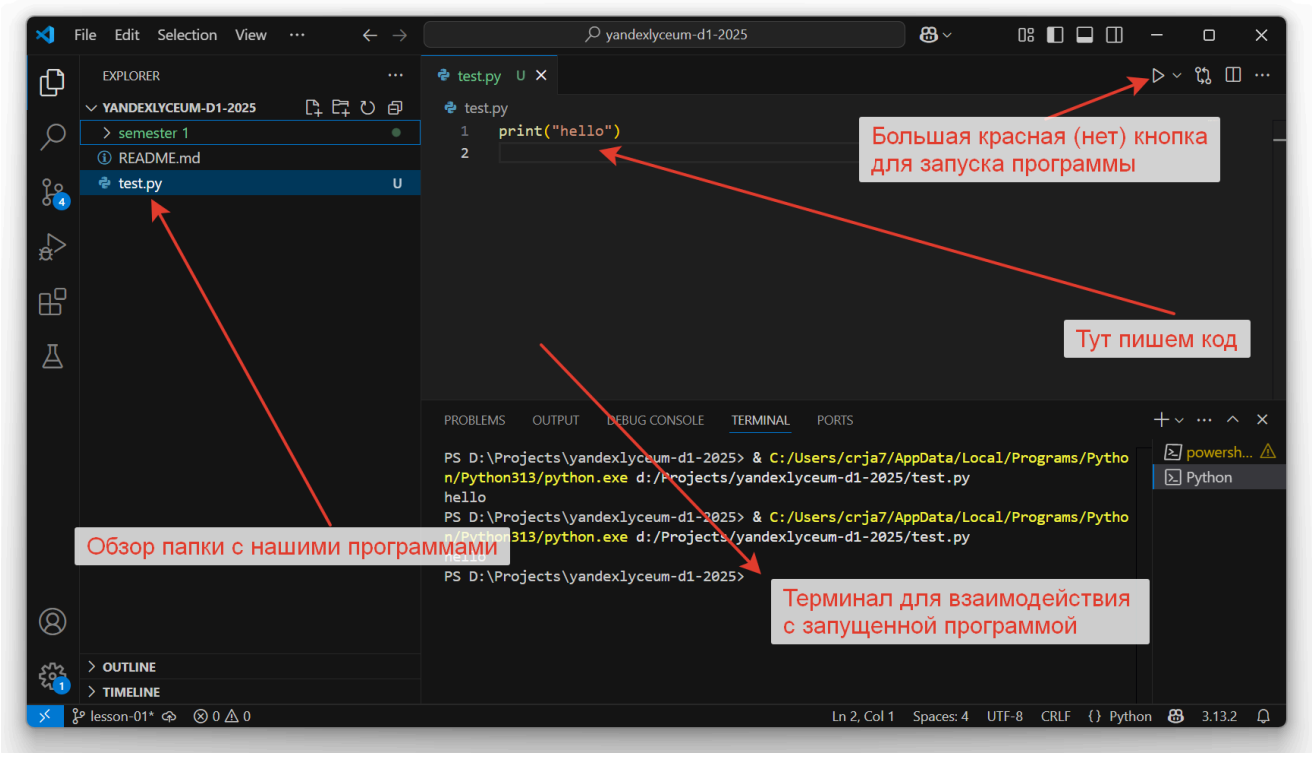
Рекомендуем создать на компьютере каталог, в котором вы будете хранить все свои программы. Назовите его «YandexLyceum», «Код_Апокалипсиса» или «Хлам_который_никто_не_проверит». Главное — не раскидывайте файлы, как носки по углам. Конечно, лучше называть каталог латинскими буквами, потому что иначе может возникнуть проблема с некоторыми сторонними библиотеками. Но так как модули, которые мы рассматриваем в этом курсе, «дружат» с русскими буквами, можете дать волю фантазии.

3. Открываем в VSCode каталог, в котором вы планируете работать. Когда вы создадите свой первый `.py`-файл, среда самостоятельно предложит вам установить все необходимые надстройки для комфортной работы. Соглашайтесь :)

4. Если что-то пошло не так, то в левой боковой панели перейдите на `Extensions` (или нажмите комбинацию клавиш `ctrl + shift + x`) и поставьте следующие плагины:

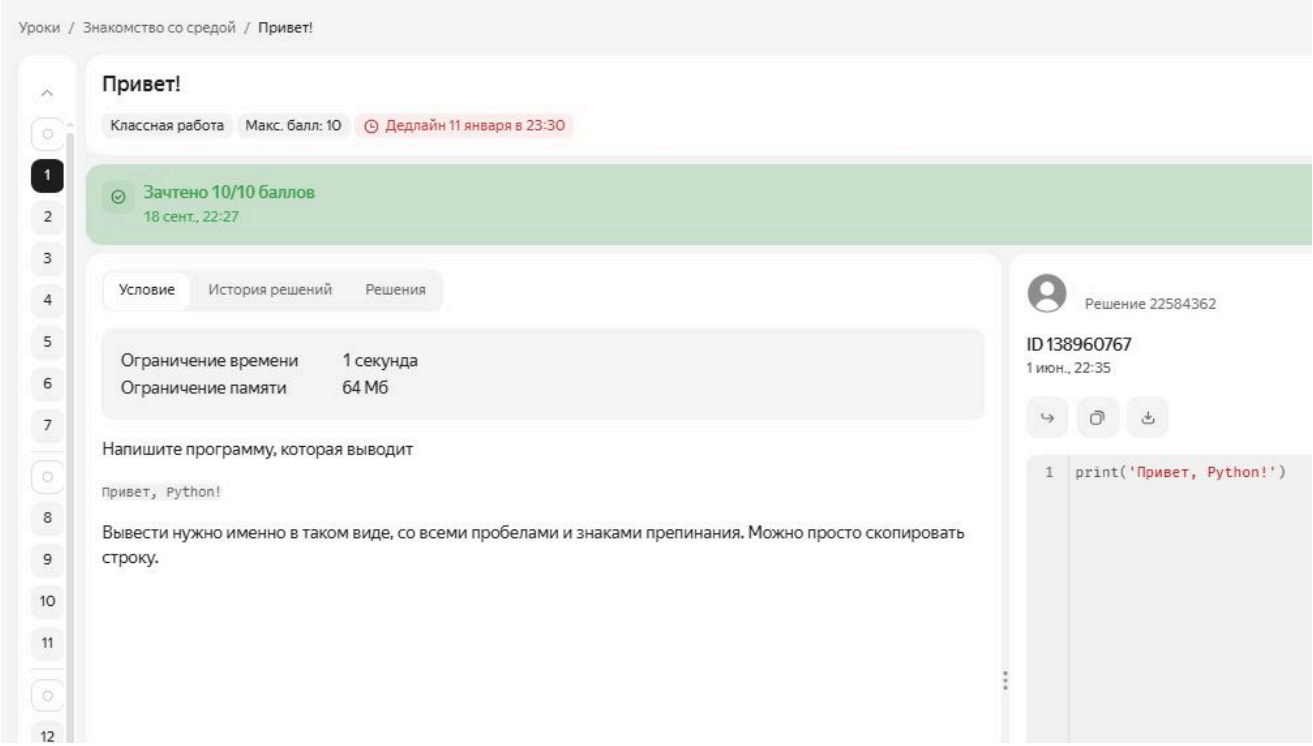
- Python (от компании Microsoft) — без него как без рук. Ну или без автодополнения кода.
- Black Formatter (тоже от Microsoft) — для добавления возможности автоматического форматирования кода по формату PEP8 (об этом чуть ниже).

Этого достаточно для начала работы, хотя для Python есть ещё огромное количество всяких расширений. Некоторые из них делают жизнь проще, а некоторые веселее — (например, расширение Power Mode), тут большой простор для экспериментов. *Совет параноика*: не качайте плагины от «Вася Пупкин.Dev». Лучше доверьтесь тем, у кого >1M установок и галочка Verified. Иначе ваш код может внезапно начать майнить биткоины.



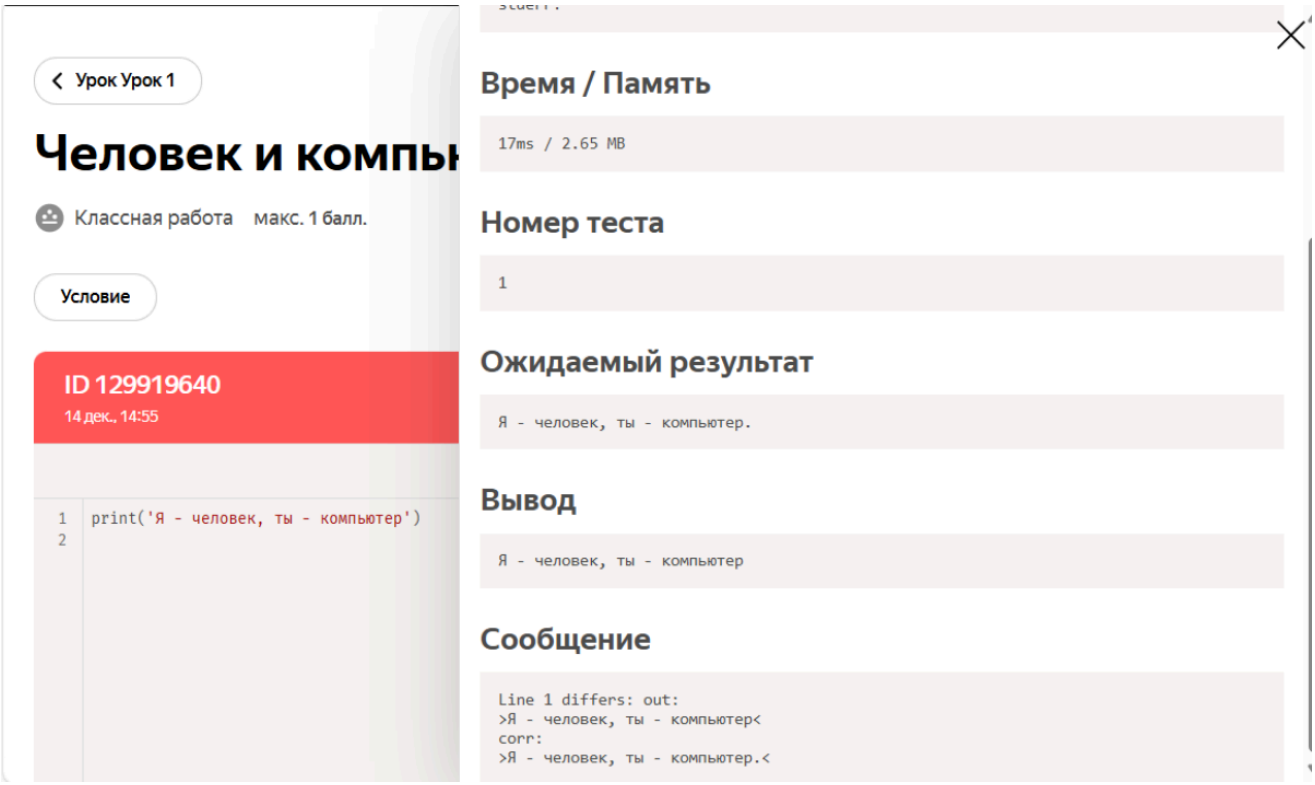
5. Задачи проверяются тестирующей системой. Для того чтобы сдать задачу на проверку, нужно открыть редактор, написать, вставить или загрузить из файла код и отправить его на проверку.

Если всё сделано правильно, полоса над редактором кода окрасится в приятный зелёный цвет, выставятся баллы за задачу, а в окошке чата задачи появится вердикт «OK» и выставленные баллы.



Но если что-то пойдёт не так, то как узнать, что именно? Для этого можем обратиться к вердикту. Заодно посмотрим, как загрузить файл, написанный не в LMS, а в другой среде программирования.

Нажав на табличку со словом «вердикт» в чате задачи, можно посмотреть, какой вывод ожидала система и что вывела ваша программа (похоже, забыли поставить точку).



Самые частые вердикты с ошибками, которые вы будете видеть:

- Вердикт «неверный ответ» — значит, ваш код работает, но делает не то. Например, вместо «Hello, World!» вывел «Go away!».
- Вердикт «ошибка компиляции или оформления кода» — вы написали «pritn» вместо «print» или ваш код не соответствует стандарту PEP8. Да, робот — граммар-наци.

📜 Да кто он такой, этот ваш PEP8?

Любой язык — хоть русский, хоть язык программирования — это как игра в Lego. Есть кирпичики (слова/команды) и правила сборки (синтаксис). При этом есть правила строгие и нестрогие:

- Строгие правила, например: если собрать кубики в рандомном порядке, получится не замок, а абстрактное «искусство».
- Нестрогие правила: можно писать x=5 или x = 5. Программа запустится, но во втором случае коллеги не спросят: «Ты вообще в школе учился?»

Синтаксис Python — это как грамматика. Нарушил — программа не запустится. PEP8 — как правила этикета. Нарушил — код будут читать, как расшифровывают почерк врача.

PEP8 — это свод законов, как писать код, чтобы:

- **Вы** через месяц поняли, что написали.
- **Коллеги** не подумывали о ритуальном сожжении вашего кода.

Мы будем вставлять правила оформления той или иной синтаксической конструкции по ходу изучения. Без выполнения этих правил проверяющая система задачи не примет.

К счастью, VSCode (а конкретно плагин Black Formatter) придёт нам на помощь, достаточно нажать `Shift + Alt + F`, чтобы код отформатировался по PEP8 сам. (В Linux — `Ctrl + Shift + I`, в MacOS — `Shift + Option + F`)

🖨️ Print: «Мама, я в консоли!»

Хотите, чтобы программа заговорила? Выводила результаты? Ругалась на пользователя? Команда `print()` — ваш инструмент.

Python — регистрозависимый язык, то есть для него имеет смысл, большая буква или маленькая. Необходимо писать `print` только маленькими буквами.

Как это работает:

- Пишете `print(` — это как взять мегафон в руки.
- Внутрь скобок кладёте то, что нужно выкрикнуть: текст, числа, смайлики.
- Закрываете открытую скобку.
- Запускаете программу.
- Смотрите на результат в терминале.

Если хотим вывести какой-то текст, указываем его внутри кавычек. Кавычки могут быть как одинарными, так и двойными. Главное, чтобы текст начинался и заканчивался кавычками одного типа. И лучше использовать один тип кавычек по всей программе.

Создайте в среде новый файл и напишите в нём команду вывода на экран слова «Привет!» Команда будет выглядеть так:

```
print('Привет!')
```

Обратите внимание: слова выделены разными цветами. В IDE используется цветовое выделение синтаксиса. Все стандартные команды и функции — одним цветом, текстовые данные — другим, числа — третьим (при этом цвет зависит от темы оформления, которую вы выбрали).

Готово, теперь можно нажать на кнопку Play в правом верхнем углу IDE и наслаждаться выводом программы в терминале, который появится снизу.

PEP8
Избегайте использования пробелов сразу перед открывающей скобкой, после которой начинается список аргументов функции.
Правильно:

```
print('Привет!')
```

Неправильно:

```
print ('Привет!')
```

Написать или запустить неверную программу — не страшно. Что-то не понять — тоже не страшно. Даже опытные программисты иногда ошибаются и что-то не понимают. Поэтому в случае затруднений всегда смело задавайте вопросы и просите преподавателя вам помочь. Мы тут для этого и собрались.

Давайте попробуем намеренно сломать нашу программу и посмотрим, как на это отреагирует Python. Для начала допустим опечатку в имени команды `print`.

```
pront('Привет!')
```

Во-первых, мы увидим, что VSCode не узнал команду и не выделил её цветом. Если же мы запустим программу с этой ошибкой, то получим:

```
NameError: name 'pront' is not defined. Did you mean: 'print'?
```

Сам интерпретатор попробует нам подсказать, что мы сделали не так.

Это как если бы вы заказали «экспрессо», а бариста сказал: «У нас нет экспрессо. Может, эспрессо?»

Конечно, если ошибка больше, чем в один символ, например если мы вместо `print` напишем `napechatay`, то интерпретатор будет бессилен и просто скажет, что не знает такой команды.

Теперь давайте забудем закрыть кавычку у строки:

```
print('Привет!)
```

```
SyntaxError: unterminated string literal
```

Интерпретатор нам прямо говорит, что мы забыли закрыть кавычку и он не понимает, где заканчивается строка.

Читайте сообщения об ошибках от интерпретатора как дружеские сообщения от Python. Да, интерпретатор не прощает ошибок, но всегда объяснит, что не так, и готов дать вам ещё шанс!

📃 Печатаем, как профессиональная типография

Мы сделали важный первый шаг и написали первую программу, которая состоит всего из одной команды. Самое время добавить ещё команд!

Давайте сделаем так, чтобы следующая программа выводила две строчки с текстом. Пусть в первой строке будет «Привет!», а во второй — «Наша вторая программа в два раза полезнее первой!».

Для этого нам понадобится ещё одна команда `print`. Эту команду обязательно надо написать с новой строки. **Каждая новая команда пишется с новой строки!**

```
print('Привет!')
print('Наша вторая программа в два раза полезнее первой!')
```

Сохраните программу, запустите и проверьте.

Проведём смелый эксперимент: разобьём строчку во втором `print` на несколько, каждую часть напишем в своих кавычках, а сами части перечислим через запятую, вот так:

```
print('Наша', 'вторая', 'программа', 'в', 'два', 'раза', 'полезнее', 'пер
```

Как уже говорилось в начале урока, внутри круглых скобок через запятую указывается то, что нужно вывести на экран. В данном случае на экран выводится несколько строк.

Всё, что мы перечислим внутри круглых скобок команды `print` через запятую, будет выведено через пробел. Если быть совсем честными, то не через пробел, а через разделитель, который по умолчанию — пробел, но уже совсем скоро мы научимся и это подчинять нашим желаниям.

PEP8
После запятой обязательно нужно добавлять пробел (как и при написании текста на русском языке)
Правильно:

```
print('Привет', 'Яндекс!')
```

Неправильно:

```
print('Привет','Яндекс!')
```

А пока ещё несколько примеров:

```
print('123')  # -> 123
print ('Hello, world!')  # -> Hello, world!
print('Hello,', 'world!')  # -> Hello, world!
print(123)  # -> 123
print(31, 'декабря наступает Новый Год!')  # -> 31 декабря наступает Новы
print()  # -> Просто перенос строки
print(2, '+', 3, '=', 2 + 3)  # -> 2 + 3 = 5
```

📦 Переменные — память нашей программы

До этого мы выводили значения непосредственно в команде `print()`. Это удобно, если значение нам нужно только один раз и потом нигде не будет использовано. Но так почти никогда не бывает, программы выполняют с данными разные действия, прежде чем выдать результат пользователю. Тут на помощь приходят переменные.

Можно представить, что **переменная — это коробка с наклейкой**. Можете положить в неё что угодно: число, текст, список котиков.

У любой переменной есть имя и значение:

- Имя переменной — это то, как вы её назвали в своей программе. Вы можете придумать любое имя, но лучше, если оно будет отражать назначение переменной. Имя не должно начинаться с цифры и может состоять из букв, цифр и символа подчёркивания. Python поддерживает имена переменных на локальных языках, например на русском или китайском, но **не используйте никакой язык, кроме английского, пожалуйста!**

PEP8
Для именования переменных принято использовать стиль *lower_case_with_underscores* (слова из маленьких букв с подчеркиваниями). Избегайте использовать такие символы, которые могут неоднозначно трактоваться в различных шрифтах: это буква **O** (большая и маленькая) и цифра **0**, буква **I** (большая и маленькая), буква **L** (большая и маленькая) и цифра **1**. Нельзя использовать в качестве имени переменной и ключевые слова, которые существуют в языке.

- Значение переменной — то, что сохраняет в себе переменная.

```
# Хорошие имена переменных:
pi = 3.14
cats_count = 10
username = 'Иван'

# Плохие имена переменных:
cats_count = 'Иван'
z = 3.14
dfogjsojdfgsoijgf = 10
```

Знак «=» обозначает команду под названием «присваивание». Оператор присваивания присваивает значение, которое находится справа от знака равно, переменной, которая находится слева от знака равно.

PEP8
Всегда окружайте оператор присваивания одним пробелом с каждой стороны:

Правильно:

```
bird = "Тук-тук"
```

Неправильно:

```
bird="Тук-тук"
```

Мы присваиваем значение переменной и потом всегда сможем получить это значение по её имени.

```
parrot_name = 'Кеша'
print('Мы купили попугая и назвали его', parrot_name)
print(parrot_name, 'умеет говорить несколько фраз')
```

Мы можем в любой момент времени изменить значение переменной на другое.

```
cat_name = 'Пудинг Иванович'
print('Моего кота зовут', cat_name)
cat_name = 'Поганец'  # После выполнения этой строчки по имени cat_name б
print('Но для друзей он —', cat_name)
```

Так как Python — язык с динамической типизацией, нам не надо задумываться, что за значение мы хотим положить в переменную и какое там лежало раньше. Кроме того, оператор присваивания сообщает переменной то или иное значение независимо от того, была ли эта переменная введена раньше.

Если у нас имеется переменная, мы можем делать с её значением всё что угодно, например присвоить другой переменной:

```
hello = 'Здравствуйте.'
hello2 = hello
print(hello2)
```

⌨️ Команда input() — налаживаем диалог с пользователем

Пока что наши программы напоминали заученные монологи: они выводили в мир заранее прописанные фразы, как робот-лектор. Но настоящая магия начинается, когда программа научится слушать.

```
print('Как тебя зовут?')   # Программа спрашивает.
name = input()             # Программа замолкает и ждёт...
print('Привет,', name)     # Программа реагирует на ответ.
```

Запустите этот код. Если в консоли появилась тишина после вопроса — это не баг. Это программа вежливо ждёт, пока вы не введёте имя и не нажмёте `Enter`.

Как это работает:

- `input()` — это команда-ловушка. Когда программа до неё доходит, она замирает и смотрит в окно терминала, как кот, ждущий мышь.
- Пользователь печатает что-то (например, «Бэтмен») и жмёт `Enter`.
- Всё, что было введено, подменяет собой `input()`. Теперь `name = 'Бэтмен'`.

Так можно, но мы так делать не будем: вы можете написать `input('Как тебя зовут? ')` без отдельного `print()`. Это как задать вопрос прямо в момент «взятия микрофона», но при этом не будет перевода на следующую строчку (без дополнительной хитрости), поэтому давайте договоримся, что будем всё выводить с помощью `print`, а с помощью `input` только вводить.

```
name = input('Как тебя зовут? ')  # Всё в одной строке
print('Привет,', name)
```

Ещё примеры:

```
print('Какая твоя любимая еда?')
meal = input()
print('Да.', meal, '— это вкусно.')
```

Обратите внимание: интерпретатор ждёт, что пользователь что-то введёт с клавиатуры ровно столько раз, сколько команд `input()` встречается в программе. Каждый `input()` завершается нажатием `Enter` на клавиатуре.

```
print('Как тебя зовут?')
name = input()  # Ждём, пока пользователь введёт имя.
print('Привет,', name)
print('А какая твоя любимая еда?')
meal = input()  # Ждём, пока пользователь введёт любимое блюдо.
print('Да.', meal, '— это вкусно.')
```

Вне зависимости от того, что ввёл пользователь, `input()` **всегда возвращает строку**, то есть даже если на вход пришло число 42, то в переменную будет записана строка «42». Что с этим делать, мы уже очень скоро узнаем.

📖 Кавычки — сколько их? Одна? Две? Три?

Мы уже говорили, что **строковые значения должны быть заключены в кавычки**, иначе интерпретатору будет невозможно их отличить от имён переменных или от команд самого языка. Это могут быть:

- **Одинарные: `'Привет'`**
- **Двойные: `"Привет"` Разницы нет, но стиль лучше соблюдать единый.**

Но можно ещё использовать три одинарные или три двойные кавычки для многострочных строковых данных:

```
poem = """Розы красные,
Фиалки синие,
Я пишу код,
И мне хорошо."""
```

📏 Длинные строки — как не потеряться в тексте

PEP8
Длина строки с кодом не должна быть более 79 символов. Это правило придумали во времена маленьких мониторов, поэтому сейчас почти всегда делается послабление. В нашей проверяющей системе максимальная длина строки — 110 символов.

Но что, если мы хотим вывести текст, который точно длиннее 110 символов? Не надо паниковать: для решения этой проблемы есть несколько способов:

- Допустимый, но нежелательный **перенос через `\`**:

```
long_text = "Это очень длинная строка, которую \
удобно разбить на две линии."
```

- Прекрасный и удобный с помощью **скобок**:

```
long_text = (
    "Этот способ даже лучше — "
    "не нужен символ переноса."
)
```

💬 Комментарии — заметки для себя и потомков

Вы, наверное, заметили, что в примерах кода есть текст после символа "#" и это не ломает работу программы. На самом деле интерпретатор игнорирует всё, что происходит после символа диеза. Так мы можем добавлять комментарии в код.

Вообще комментарии бывают нескольких видов:

- **Однострочные комментарии**, на которые вы успели насмотреться выше

```
# Это важный комментарий. Не удалять!
x = 5  # Волшебная константа.
```

- **Многострочные**:

```
'''
Многострочные комментарии пишутся именно в трёх одинарных кавычках
Не будут ошибкой и три двойные
Но так не принято в приличном обществе
'''
```

Зачем?

- Объяснить сложную логику.
- Временно отключить код (закомментировать).
- Напомнить себе, что тут происходит (через месяц всё забудется).

Не пишите очевидные комментарии вроде `x = 5 # Присвоить x значение 5`. Лучше объясните, зачем это нужно:

```
# Рассчитать дискриминант квадратного уравнения
D = b ** 2 - 4 * a * c
```

📝 Ещё пример по шагам

Разберём код:

```
a = 5
b = a  # b = 5. Нет, это не создаёт связь навеки!
a = 10
print(a + b)  # Выведет 15. Да, переменные живут своей жизнью.
```

Таблица трассировки для чайников:

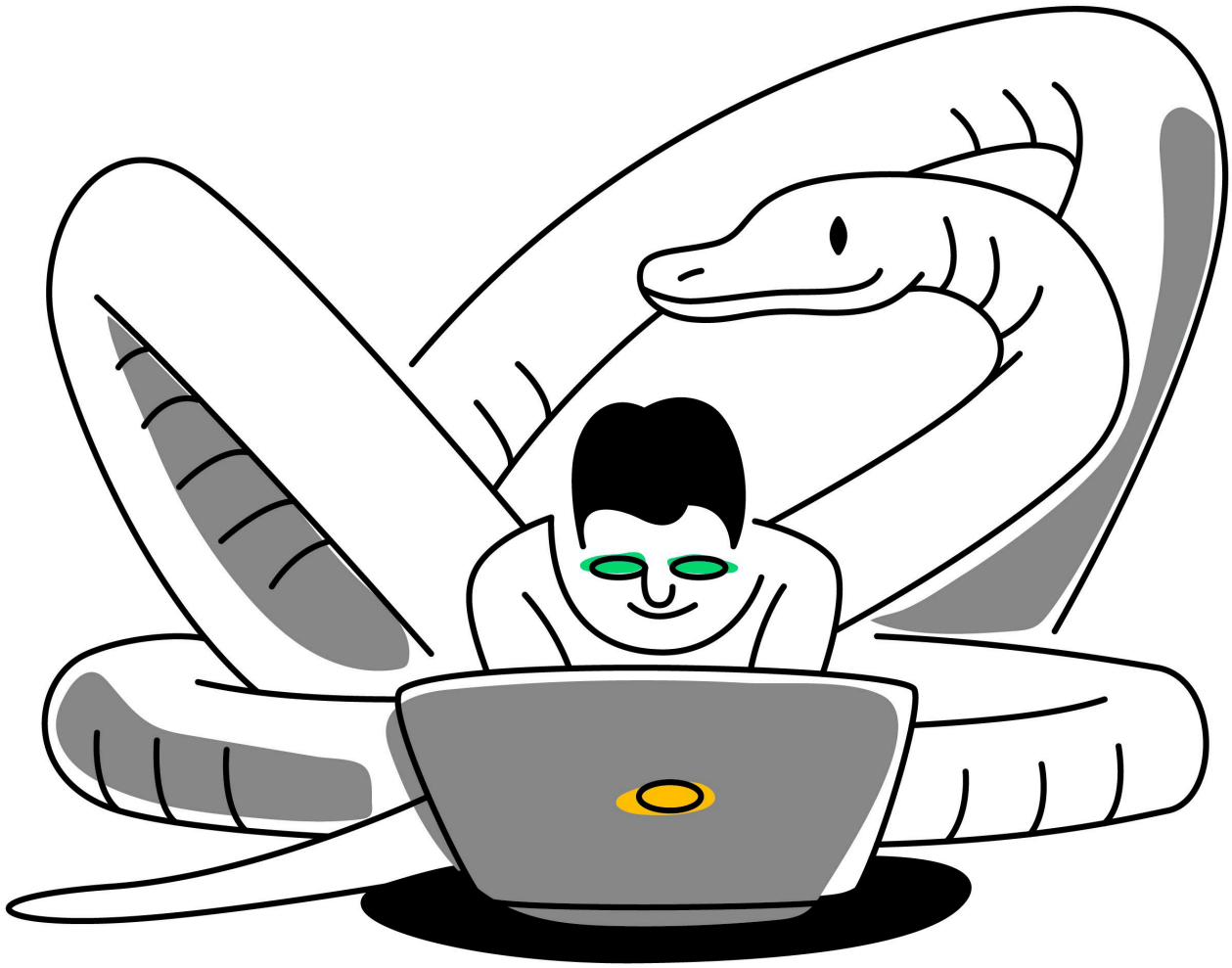
Шаг	Код	Переменная a	Переменная b	Состояние программы
1	`a = 5`	5	не определена	«А что такое b?»
2	`b = a`	5	5	«Боже, они одинаковые!»
3	`a = 10`	10	5	«a, ты предатель!»
4	`print(a + b)`	10	5	«Так, надо разобраться, что нужно вывести»
4.1	`a + b`	10	5	15. Можно удалять приложение «Калькулятор»
4.2	`print(15)`	10	5	Вывод: 15. Все в шоке.

Прежде чем выполниться, любая функция Python должна вычислить свои аргументы, поэтому если мы выполняем какие-нибудь (математические или какие-то ещё) операции в `print`, то в терминале мы увидим уже результат.

🌟 Итог: Вы уже программисты!

Поздравляем! Вы прошли первый уровень Python-вселенной:

- Настроили VSCode и теперь можете кодить на стиле.
- Узнали, как общаться с Python через `print()` и `input()`.
- Усвоили, что код должен быть красивым, как котик в смокинге (спасибо PEP8).



P. S. Если устали — поспите, работает так же эффективно, как перезагрузка компьютера.